Ministry of Higher Education and Scientific Research



International Polytechnic Private School Of Tunis

Spring BootProject

Specialty: Computer Networks and Multimedia

Developed by

Ines Anane et Asma Hammami

Bank Management System

Supervised by

Mr Hassen Lazreg

Academic Year : 2023-2024

Contents

General Introduction **1**

1 General Framework of the Project **2**

1.1 Presentation of the Project . . . 3

1.1.1 Study of the Existing System . . . 3

1.1.1.1 Description of the Existing System . . . 3

1.1.1.2 Critique of the Existing System . . . 3

1.1.2 Proposed Solution . . . 3

1.1.3 Problematic . . . 4

1.2 Specifications Document . . . 4

1.2.1 Project Objective . . . 4

1.2.2 Project Target . . . 4

1.2.3 Functional Requirements . . . 4

1.2.4 Non-Functional Requirements . . . 5

2 Specification of Requirements **6**

2.1 Candidate Requirements . . . 7

2.2 General Use Case Diagram . . . 8

2.3 System Sequence Diagrams . . . 9

2.3.1 System Sequence Diagram: "Authentication" . . . 9

3 Modeling **10**

3.1 Use Case Diagram "Account Management" . . . 11

3.1.1 Textual Description "Account Management" . . . 12

3.1.2 Object Sequence Diagram "Account Management" . . . 13

3.1.3 Activity Diagram "Manage Account" . . . 14

3.2 Use Case Diagram "Create New User Account" 15
3.2.1 Textual Description "Create New User Account" 16
3.2.2 Object Sequence Diagram "Create New User Account" 17
3.2.3 State Transition Diagram "Create New User Account" 18

4 Conception **19**

4.1 Data Conception 20
4.1.1 Final Class Diagram 20
4.1.2 Relational Schema 20
4.2 Architecture Design 21
4.2.1 MVC Architecture 21
4.2.2 Layered Architecture 22
4.2.2.1 Presentation Layer 22
4.2.2.2 Business Layer 22
4.2.2.3 Data Access Layer 22
4.2.2.4 Infrastructure Layer 23
4.2.2.5 Benefits of Layered Architecture 23

5 Implementation **24**

5.1 Introduction 25
5.2 Development Platform 25
5.2.1 Software Selection 25
5.2.2 Front-end Technologies 25
5.2.3 Back-end Frameworks 26
5.2.4 Version Control and CI/CD 26
5.3 Admin Panel: Client Management Overview 27
5.4 Client Information Editing Interface 27
5.5 Client Deletion Confirmation Process 28
5.6 Bank Account Management Dashboard 28
5.7 Bank Account Operations Overview 29
5.8 Transaction History and Account Activity 29

5.9 Deposit and Withdrawal Transaction Interface 30
5.9.1 Deposit Transaction Interface 30
5.9.2 Withdrawal Operation Interface 31
5.10 Client Credit Management Dashboard 32
5.11 Credit Application Submission Confirmation 32
5.12 Credit Application Submission Confirmation 33
5.13 Credit Application History and Status Tracking 33
5.14 Bank Card Management System 33
5.15 New Bank Card Creation Interface 34
5.16 Bank Card Creation Confirmation" 34

General Conclusion **35**

List of Figures

2.1 General Use Case Diagram 8

2.2 System Sequence Diagram: "Authentication" 9

3.1 Use Case Diagram "Account Management" 11

3.2 Object Sequence Diagram "Account Management" 13

3.3 Activity Diagram "Manage Account" 14

3.4 Use Case Diagram "Create New User Account" 15

3.5 Object Sequence Diagram "Create New User Account" 17

3.6 State Transition Diagram "Create New User Account" 18

4.1 Final Class Diagram 20

4.2 MVC Architecture 21

4.3 Layered Architecture 23

5.1 Spring Boot 26

5.2 Spring Data JPA 26

5.3 GitHub and CI/CD 26

5.4 Admin Interface: Client Data Tablef 27

5.5 Edit Client Details Form 27

5.6 Client Removal Confirmation Dialog and List 28

5.7 Bank Account List with 'View Transactions' and 'Create New Account' Options 28

5.8 Bank Account Management Table with 'View Operations' Action 29

5.9 Bank Account Operations Overview 29

5.10 Bank Account Transaction History with Deposit/Withdrawal Options 30

5.11 Bank Account Operations Overview 30

5.12 Bank Account Deposit Form with Confirmation Options 31

5.13 Account Withdrawal Form with Amount Entry 31
5.14 Client Credit Management Dashboard 32
5.15 Successful Credit Request Receipt with Summary 32
5.16 Successful Credit Request Receipt with Summary 33
5.17 Credit Request Log with Approval/Rejection Status 33
5.18 Bank Card Management Interface with Card Creation Option 33
5.19 Bank Card Registration Form 34
5.20 Successful Bank Card Generation Notification 34

List of Tables

1.1 Project Target 4

2.1 System Requirements for Bank Management System 7

3.1 Textual Description "Account Management" 12

3.2 Textual Description "Create New User Account" 16

5.1 Software Used During Development 25

Introduction

The Bank Management System is a comprehensive web application designed to simplify and streamline banking operations for both customers and bank administrators. Built with Spring Boot and backed by a robust database architecture, the system offers a secure, user-friendly platform for managing essential banking services.

For customers, the system provides an intuitive interface to create and manage accounts, check balances, and perform core transactions. Whether it's opening a new account, choosing between different account types (like savings or current), or viewing account details, users can carry out operations efficiently from the web portal. Security is a top priority, ensuring that sensitive user data and financial information are protected through modern authentication methods.

For bank staff and administrators, the application delivers powerful tools for user management, account tracking, and transaction monitoring. By automating routine tasks and offering centralized access to customer data, the system significantly reduces administrative workload and improves operational transparency.

The platform is built with scalability in mind, allowing for the integration of additional modules like loan processing, credit card management, and transaction analytics. Future improvements may include real-time notifications, mobile banking support, and advanced fraud detection mechanisms.

By modernizing core banking functions, this project aims to provide a better experience for users while ensuring reliability and performance. With its strong foundation and room for expansion, the Bank Management System serves as a step toward digital transformation in the banking sector.

Chapter 1

General Framework of the Project

Introduction

This project aims to modernize banking operations through a user-friendly and secure web application. The goal is to simplify essential banking processes such as account creation, transaction tracking, and user management. By centralizing these functionalities on a single digital platform, the system enhances communication, improves efficiency, and ensures a seamless experience for both customers and bank staff.

1.1 Presentation of the Project

1.1.1 Study of the Existing System

1.1.1.1 Description of the Existing System

Traditional banking systems often rely on outdated software or fragmented tools, leading to a lack of integration across departments. Customers may need to visit physical branches for tasks like opening an account, viewing transaction history, or changing account settings. Internally, bank staff manage operations manually or with systems that do not communicate efficiently, which results in longer processing times, increased chances of error, and reduced customer satisfaction.

1.1.1.2 Critique of the Existing System

The current system presents several drawbacks. It lacks automation and centralization, making routine tasks time-consuming. Manual procedures can lead to errors such as incorrect data entries or delayed service delivery. From a customer perspective, there's limited access to self-service features, and security may be compromised due to weak or inconsistent digital practices. These factors reduce productivity for bank employees and convenience for clients.

1.1.2 Proposed Solution

To overcome these issues, we propose the development of a complete web-based banking platform using Spring Boot. This system will allow customers to open and manage accounts online, choose account types (e.g., savings, current), and securely perform key transactions. On the administrative side, the application will provide tools for monitoring user activity, managing account data, and maintaining financial records. This solution aims to optimize workflows, reduce manual errors, and enhance the banking experience for all users.

1.1.3 Problematic

These challenges lead to the key question: *"How can we develop a secure, modern digital banking solution that streamlines account and transaction management while improving user experience?"*

1.2 Specifications Document

1.2.1 Project Objective

The main objective of this project is to build a responsive and secure application that supports core banking activities such as user registration, account management, and transaction monitoring.

1.2.2 Project Target

System Administrator	Responsible for managing the platform's maintenance, applying updates, and ensuring database and system security.
Bank Staff	Use the system to monitor client activities, manage accounts, and support customer service operations.
Customers	Can register, create and manage their accounts, view transaction history, and perform secure banking actions.

Tableau 1.1: Project Target

1.2.3 Functional Requirements

- **Authentication**: Secure login and session management for different user roles.
- **Account Management**: Enable users to create accounts, choose account types, and manage personal information.
- **Transaction Handling**: Allow users to perform and view deposits, withdrawals, and transfers.
- **User Dashboard**: Provide users with a dashboard summarizing their accounts and recent activities.
- **Admin Panel**: Allow bank staff to view customer data, validate account requests, and monitor activity.
- **Reporting Tools**: Generate reports on accounts, transactions, and user activities for analysis.

1.2.4 Non-Functional Requirements

- **Security**: Encrypt sensitive data, use secure access controls, and implement best practices to prevent fraud.
- **Reliability**: Ensure stable performance and uptime for all users.
- **Performance**: Maintain fast response times under load and support concurrent users.
- **Scalability**: Support future features like loans, credit cards, or multi-branch integration.
- **Compatibility**: Ensure the application runs smoothly across devices and modern web browsers.

Through this chapter, we have defined the project's scope, identified weaknesses in current banking systems, and proposed a comprehensive digital solution. The next chapter will present the architecture of the proposed system and detail the implementation approach.

Chapter 2

Specification of Requirements

Introduction

Defining the system's requirements is a crucial step in the development of a successful banking application. It ensures that the system aligns with the expectations of users while providing the necessary features to carry out key financial tasks securely and efficiently. In this chapter, we outline the core requirements and design elements of our bank management system.

2.1 Candidate Requirements

Number	Requirement Description	Priority	Risk Level
1	The system should allow users (customers and staff) to register and create secure accounts.	High	Low
2	Users must be able to log in with strong authentication measures.	High	Medium
3	Customers should be able to open a bank account and choose an account type (e.g., savings, current).	High	High
4	The system should allow users to view and manage account details (balance, type, etc.).	High	Medium
5	All transactions (deposits, withdrawals, transfers) must be tracked securely and transparently.	High	High
6	Bank staff should be able to access user accounts and monitor activities for security and support.	High	Medium
7	The system should ensure that personal and financial data is protected according to security standards.	High	High
8	Reports on transactions, account activities, and customer trends should be available to administrators.	Medium	Medium

Tableau 2.1: System Requirements for Bank Management System

2.2 General Use Case Diagram

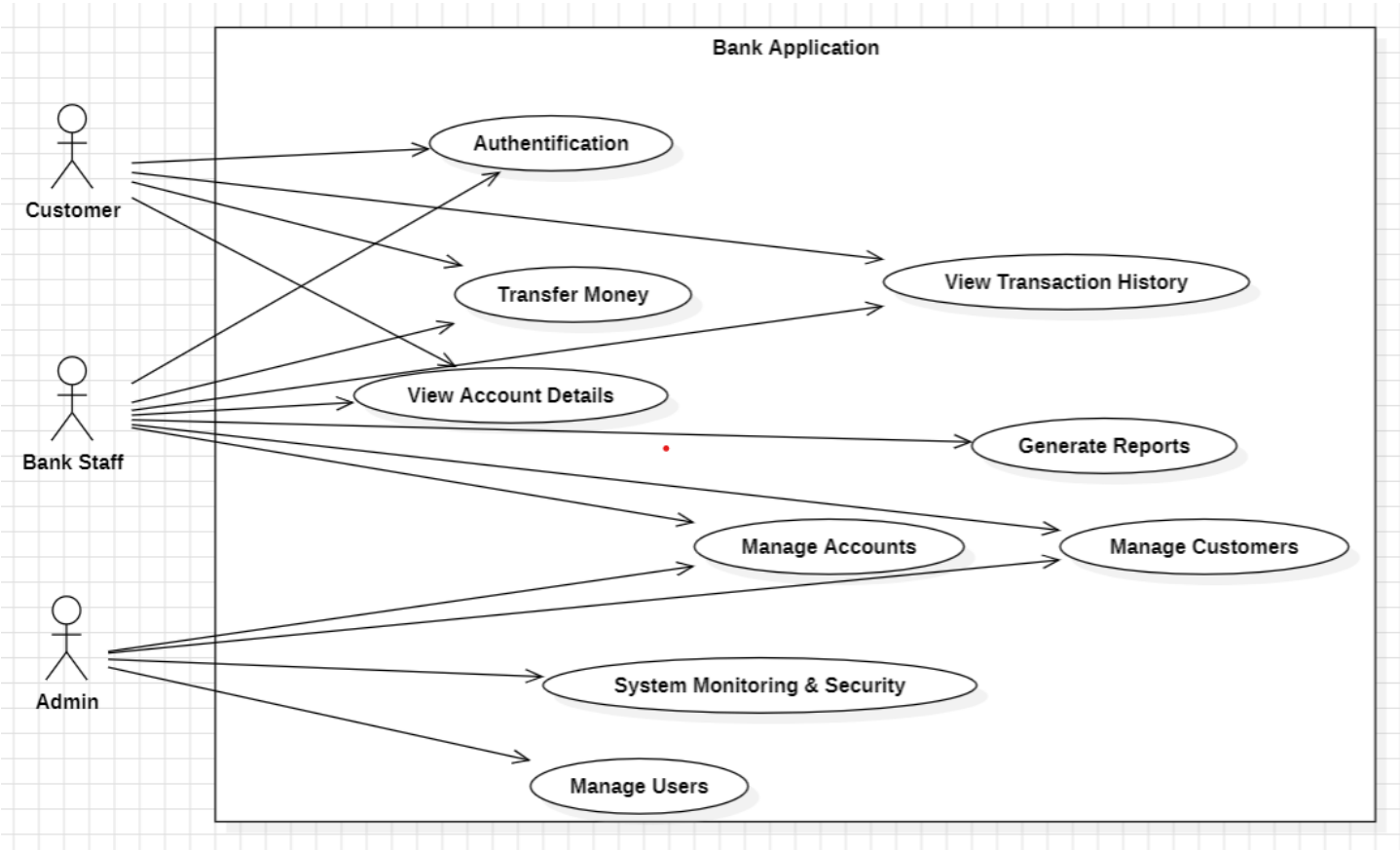


Figure 2.1: General Use Case Diagram

2.3 System Sequence Diagrams

2.3.1 System Sequence Diagram: "Authentication"

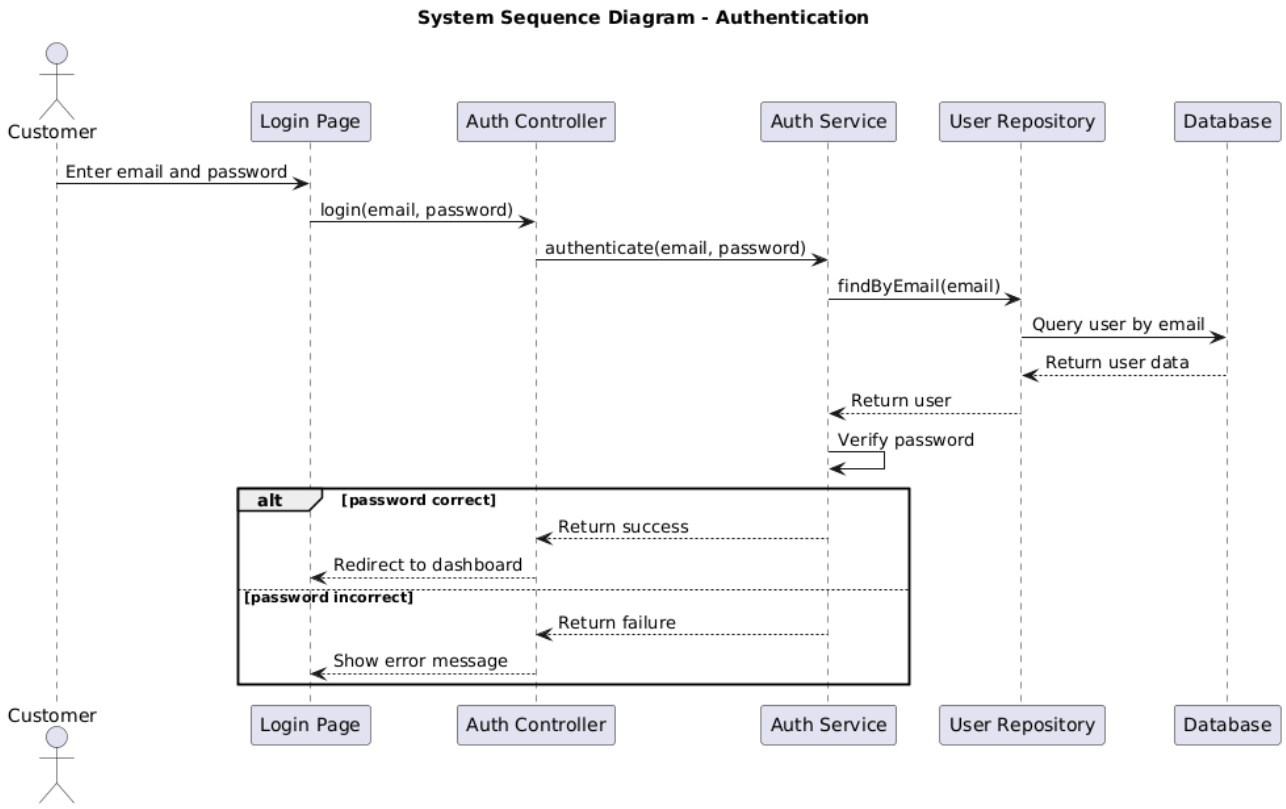


Figure 2.2: System Sequence Diagram: "Authentication"

Conclusion

Clearly defining the requirements of a banking management system is essential to build a secure and effective application. By outlining user roles, identifying key features, and planning system behavior through diagrams, we create a strong foundation for implementation. This strategic preparation ensures a system that is reliable, user-friendly, and capable of evolving to meet future financial service needs.

Chapter 3

Modeling

Introduction

Needs analysis is one of the most important phases in the software life cycle. This chapter presents a detailed and clear description of the expectations of users and features that the system will provide to respond to them, using Use Case diagrams, text descriptions, and graphs.

3.1 Use Case Diagram "Account Management"

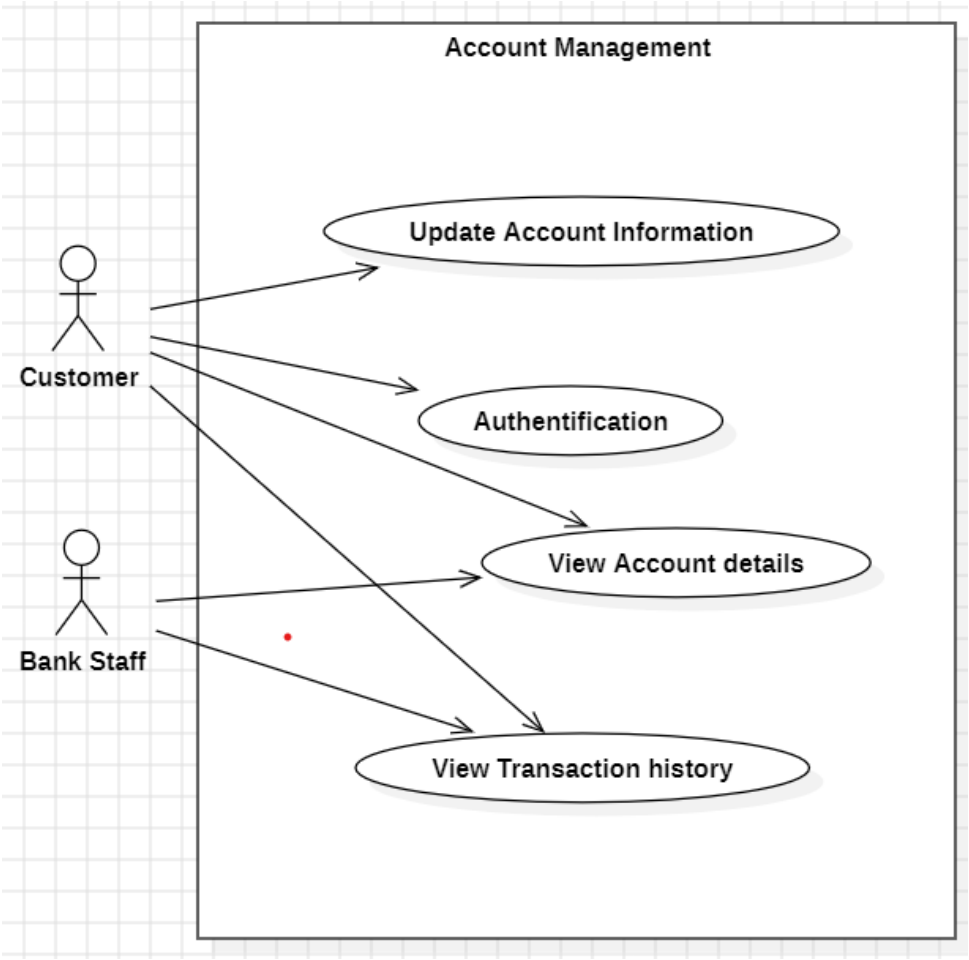


Figure 3.1: Use Case Diagram "Account Management"

3.1.1 Textual Description "Account Management"

Objective	Manage user accounts for clients, including viewing balance, transaction history, and managing account settings.
Description	This use case allows a customer to manage their account, including viewing balance, transaction history, and updating account information.
Actors	Customer, Bank Staff
Pre-condition	The customer is authenticated and has at least one account.
Post-condition	The account information is updated, and the transaction history is available.
Nominal Scenario	1. The customer logs in to the system. 2. The customer selects an account to view. 3. The system displays account details and balance. 4. The customer selects an option to view transaction history. 5. The system displays the transaction history. 6. The customer confirms any updates and submits changes if needed. 7. The system saves the changes and displays a confirmation message.
Alternative Scenario	* If the customer cannot view the transaction history, the system notifies the customer of the issue and suggests alternatives. * If there is an issue updating the account details, the system prompts for retry or error resolution.

Tableau 3.1: Textual Description "Account Management"

3.1.2 Object Sequence Diagram "Account Management"

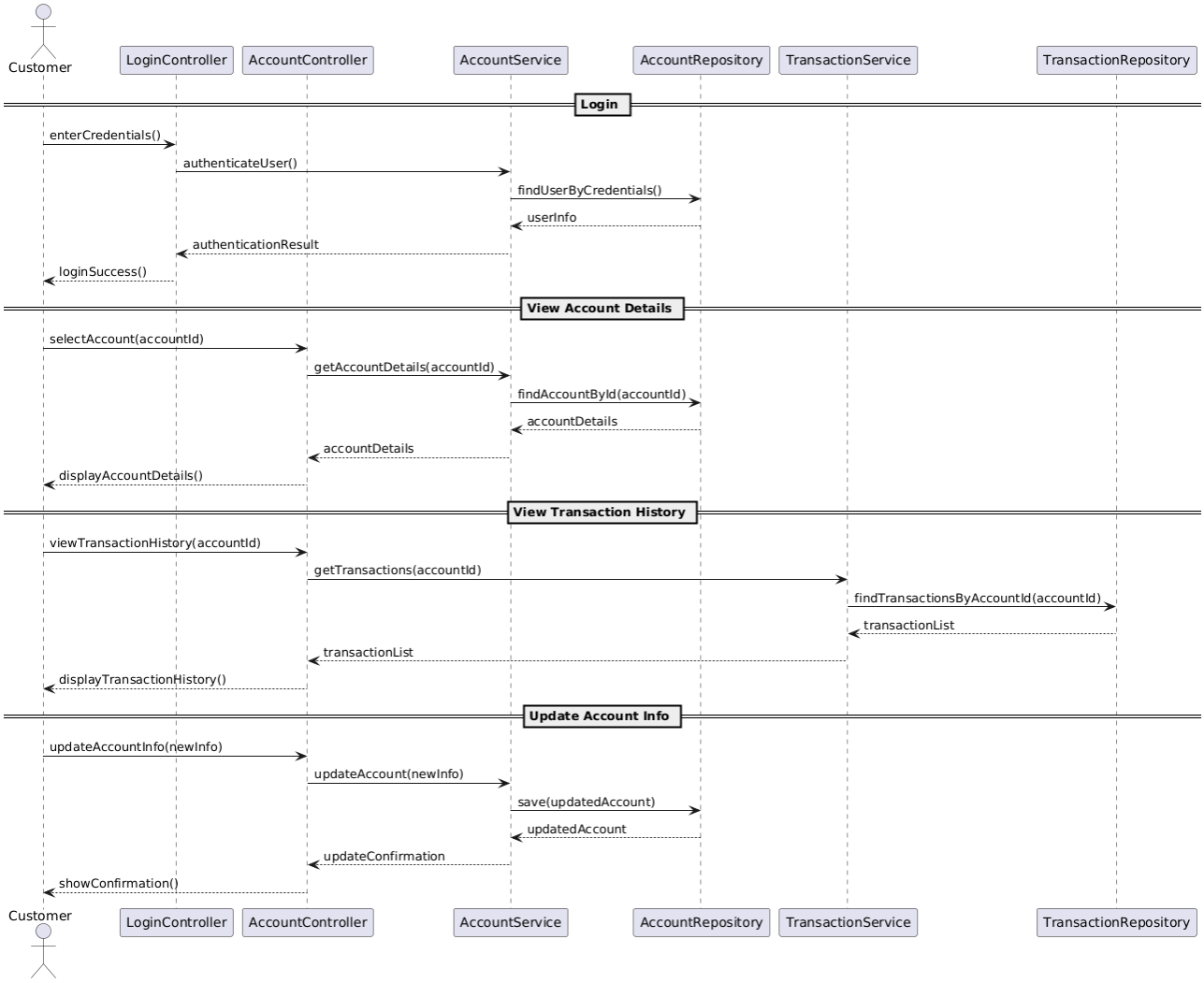


Figure 3.2: Object Sequence Diagram "Account Management"

3.1.3 Activity Diagram "Manage Account"

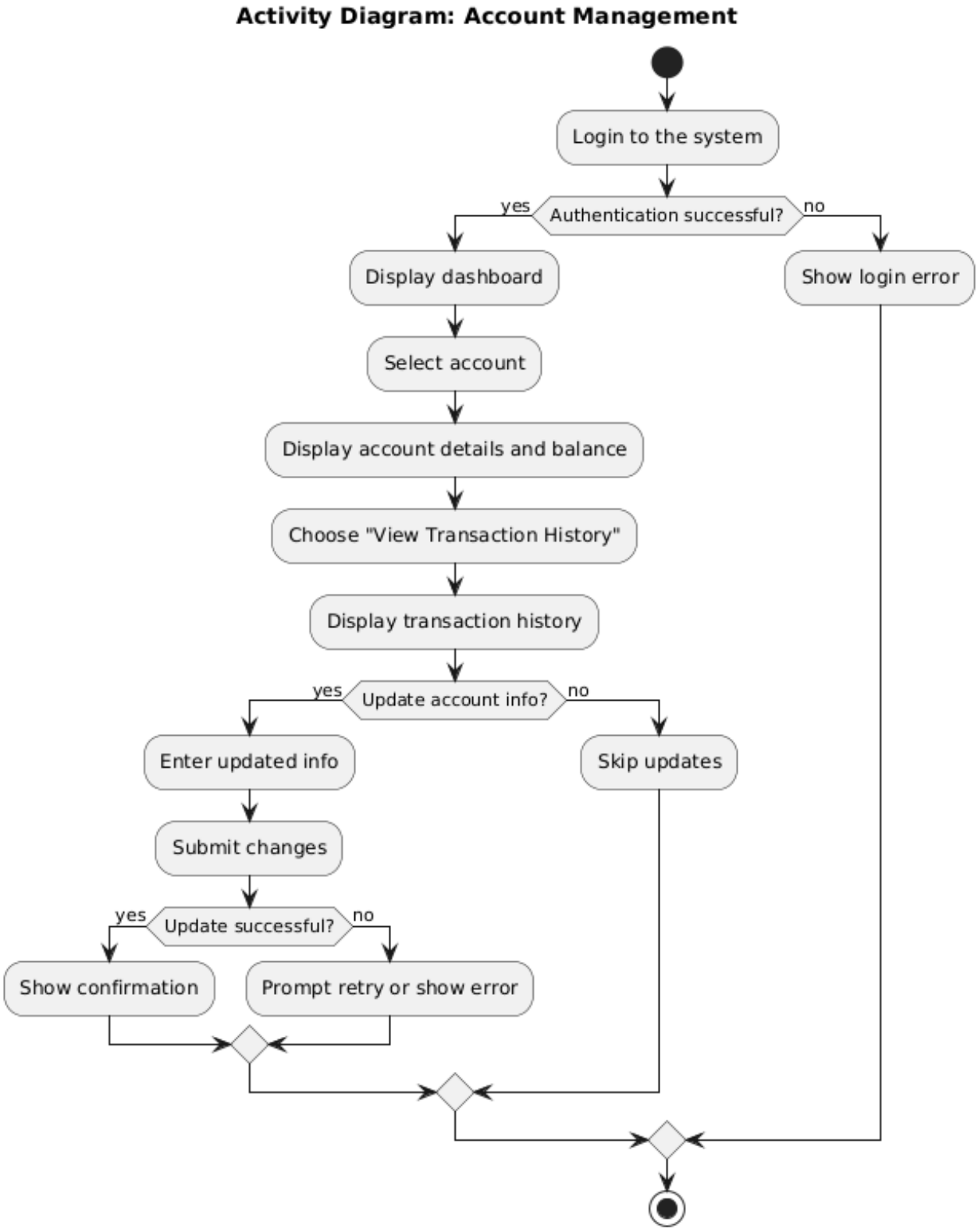


Figure 3.3: Activity Diagram "Manage Account"

3.2 Use Case Diagram "Create New User Account"

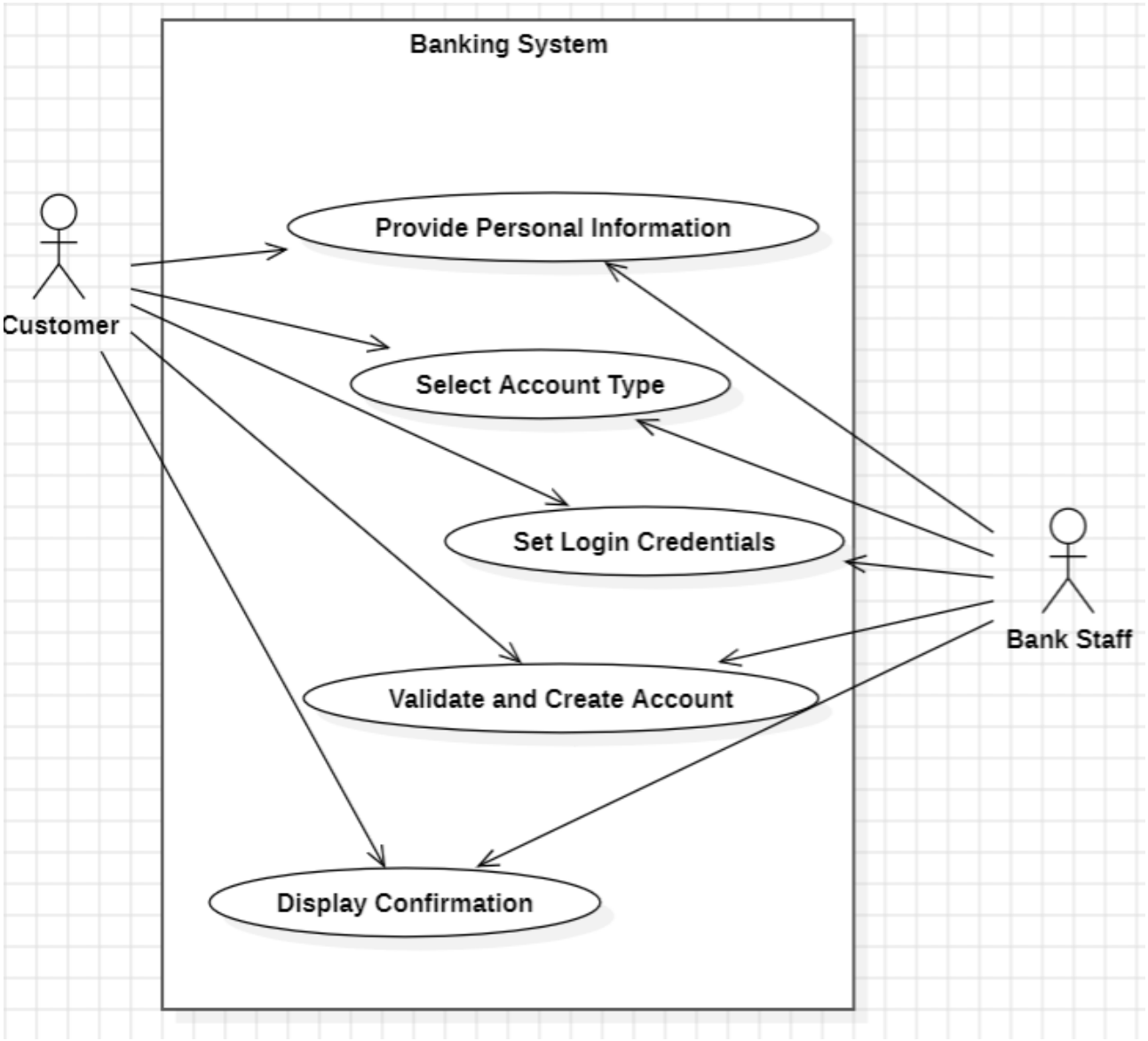


Figure 3.4: Use Case Diagram "Create New User Account"

3.2.1 Textual Description "Create New User Account"

Objective	Create a new user account for a client, allowing them to manage their bank accounts.
Description	This use case allows a customer to create a new account by providing personal details and setting up login credentials.
Actors	Customer, Bank Staff
Pre-condition	The customer must provide valid personal information and email address.
Post-condition	A new account is created for the customer, and they can log in to the system.
Nominal Scenario	1. The customer provides personal information (name, email, etc.). 2. The customer selects account type (Saving/Current). 3. The customer sets login credentials. 4. The system validates the information and creates the account. 5. The system displays a confirmation message with login instructions.
Alternative Scenario	* If the email address is already in use, the system prompts the customer to choose a different email address. * If the customer fails to provide required details, the system requests the missing information.

Tableau 3.2: Textual Description "Create New User Account"

3.2.2 Object Sequence Diagram "Create New User Account"

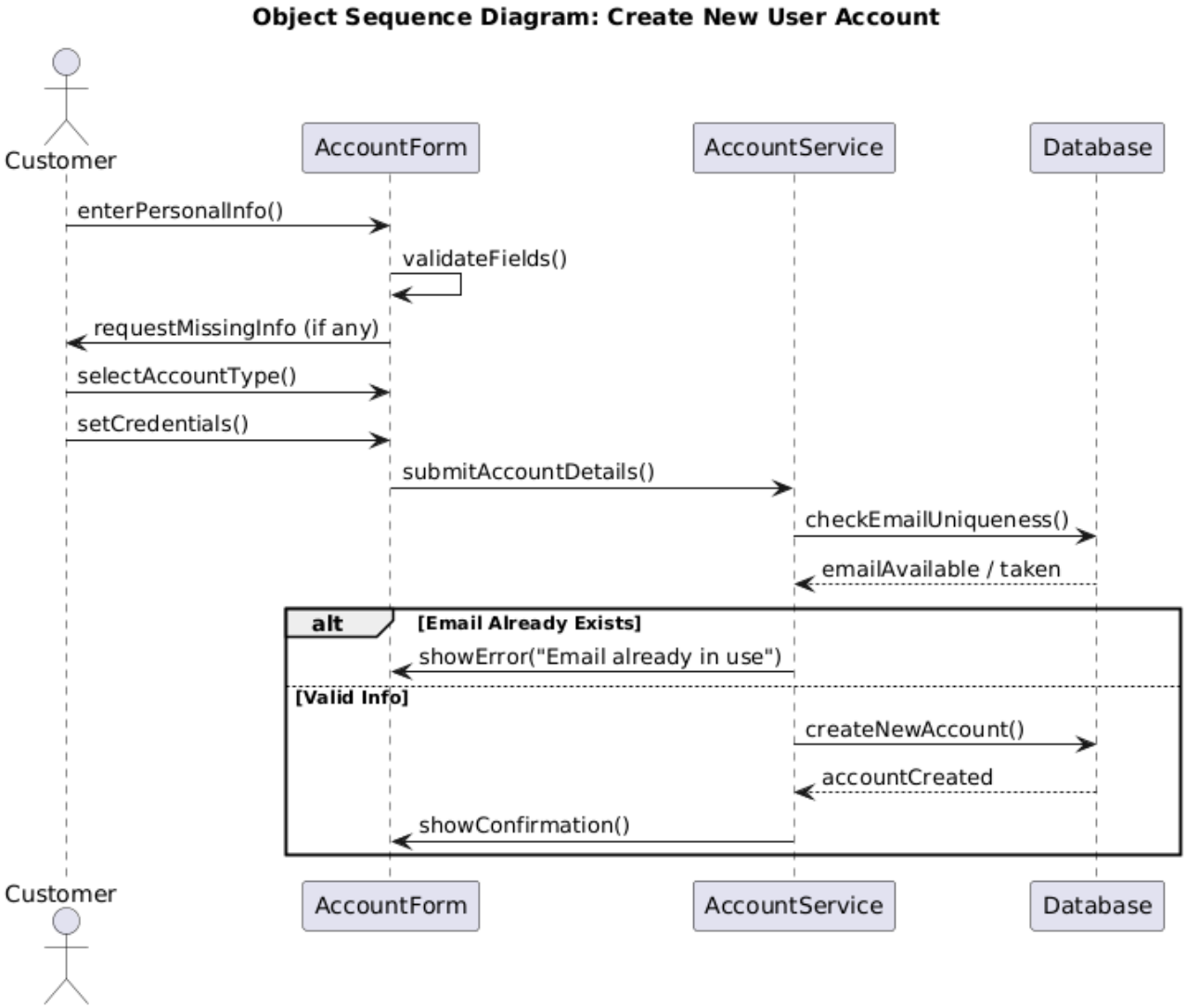


Figure 3.5: Object Sequence Diagram "Create New User Account"

3.2.3 State Transition Diagram "Create New User Account"

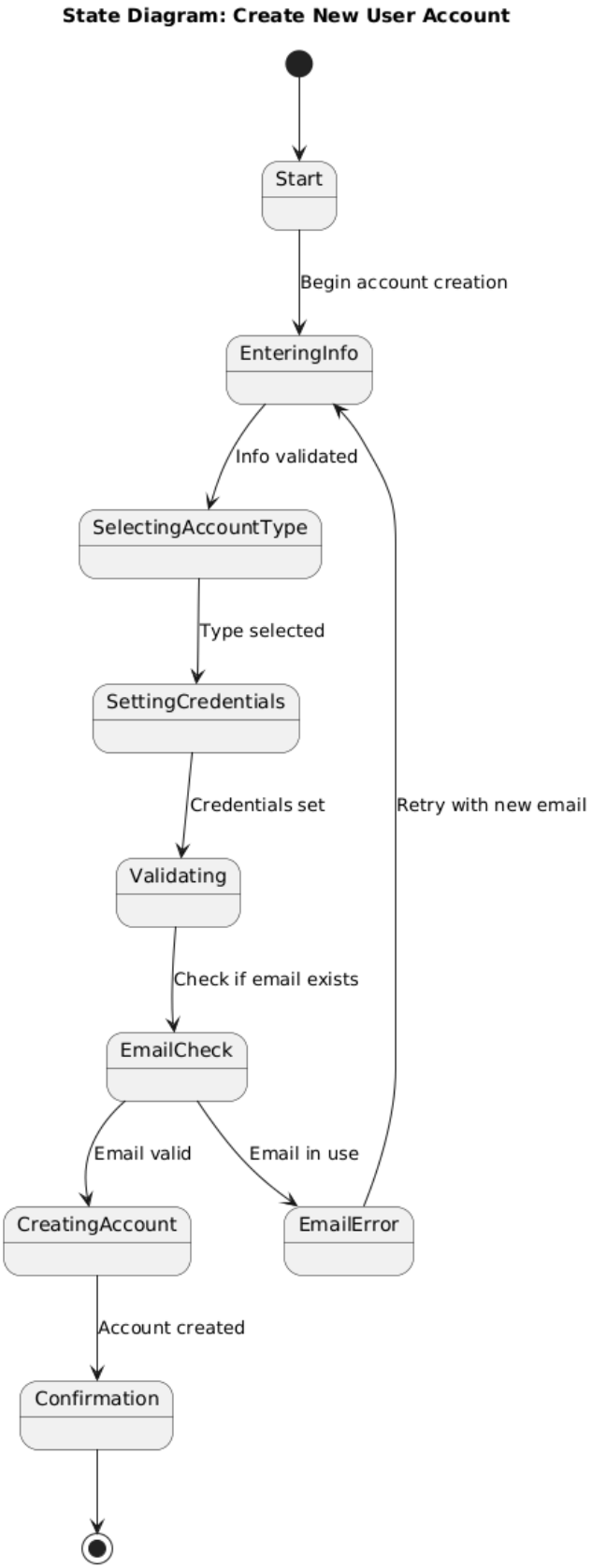


Figure 3.6: State Transition Diagram "Create New User Account"

Chapter 4

Conception

Introduction

In this chapter, we detail the design of our banking application, focusing on both the data model and the system architecture to ensure clarity, maintainability and scalability.

4.1 Data Conception

4.1.1 Final Class Diagram

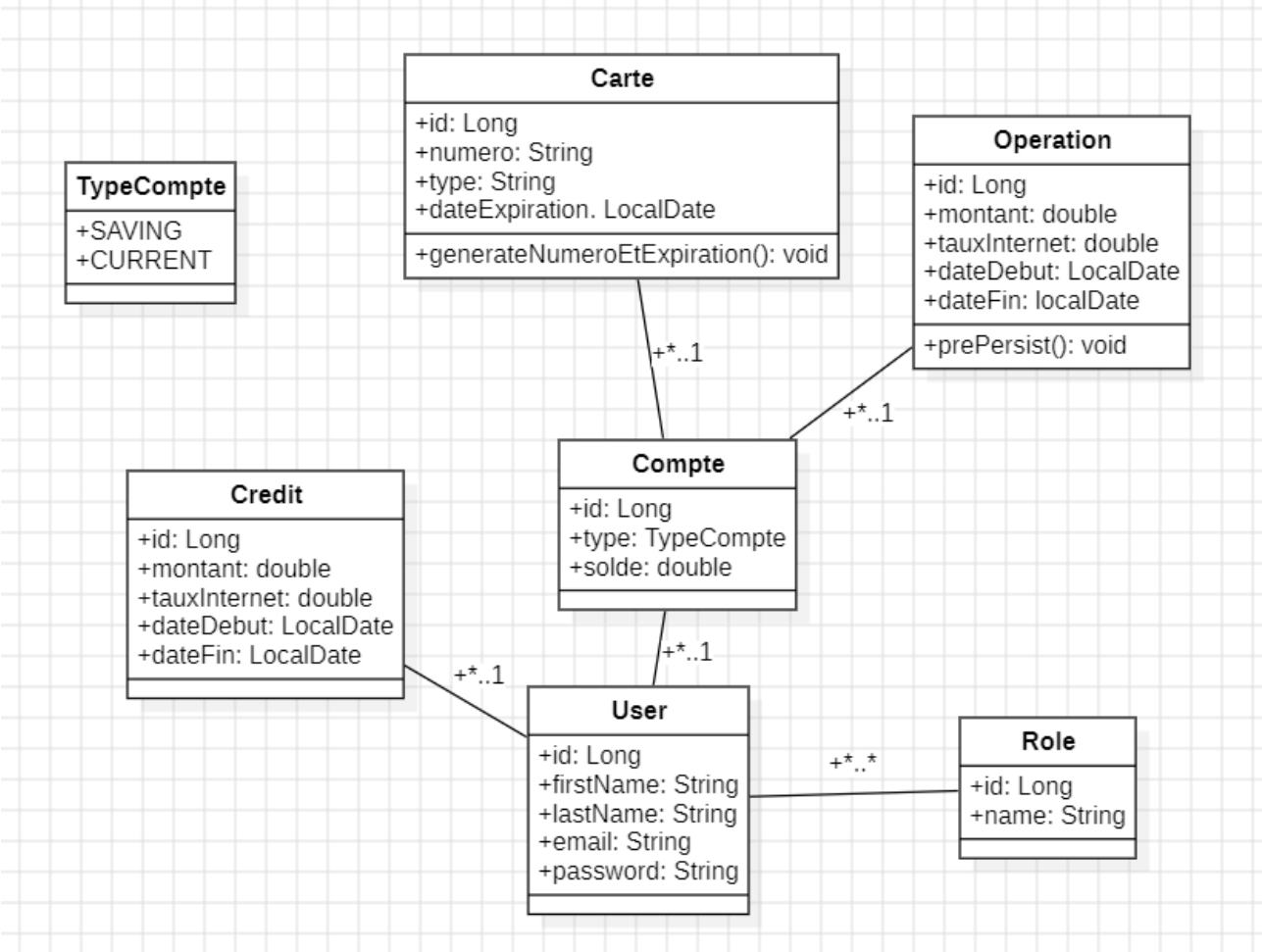


Figure 4.1: Final Class Diagram

4.1.2 Relational Schema

- **User** (Id, FirstName, LastName, Email, Password)
- **Role** (Id, Name)
- **users_roles** (#UserId, #RoleId)
- **Client** (Id, Name, Surname, Email, Phone)

- **Compte** (Id, Type, Balance, #UserId)
- **Operation** (Id, Type, Amount, Date, #CompteId)
- **Credit** (Id, Amount, InterestRate, StartDate, EndDate, #ClientId)
- **Carte** (Id, Number, Type, ExpirationDate, #CompteId)

4.2 Architecture Design

4.2.1 MVC Architecture

We adopted the MVC (Model-View-Controller) architecture due to its clear separation of concerns among data handling, user interface, and logic processing—making it well-suited for our banking application.

- **Model:** Contains the business entities (User, Compte, Carte, Credit, etc.) and manages the database structure through JPA.
- **View:** Represents the user interface. In our application, we use **Thymeleaf** to render dynamic HTML pages for users.
- **Controller:** Handles the communication between the model and the view. It processes user requests, applies business logic, and returns the appropriate view.

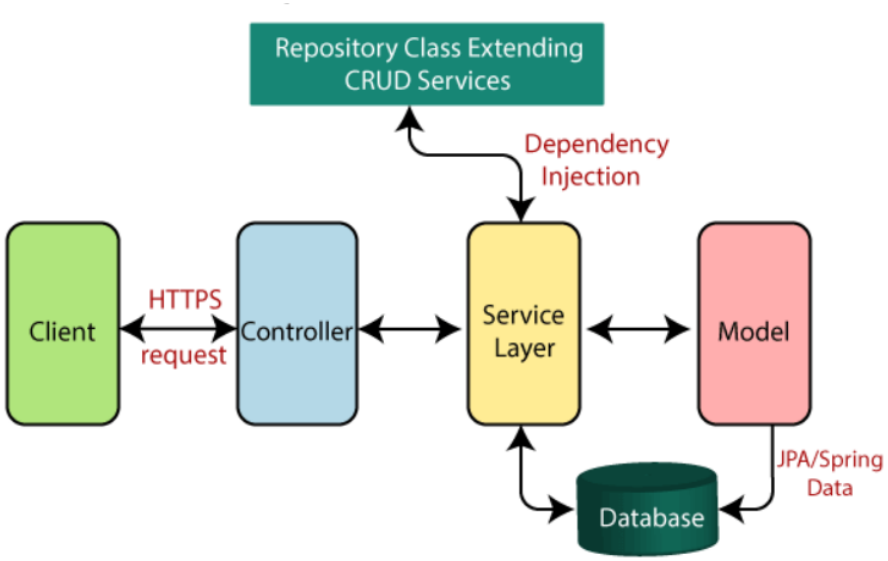


Figure 4.2: MVC Architecture

4.2.2 Layered Architecture

To ensure maintainability and scalability, our application follows a layered architecture comprising four main layers:

4.2.2.1 Presentation Layer

This layer manages user interaction and displays data. It contains web controllers and views rendered using Thymeleaf.

- Handles HTTP requests and responses.
- Validates user input and forwards it to the business layer.

4.2.2.2 Business Layer

This layer includes the core logic of the application.

- Manages operations such as creating accounts, managing credit requests, performing transactions, and generating cards.
- Contains service classes like `CompteService`, `CarteService`, `CreditService`, etc.

4.2.2.3 Data Access Layer

Responsible for interacting with the database using Spring Data JPA.

- Contains repository interfaces like `UserRepository`, `CompteRepository`, and `CreditRepository`.
- Simplifies CRUD operations and custom queries.



4.2.2.4 Infrastructure Layer

Provides technical services like security, logging, and integrations.

- Manages user authentication and authorization using **Spring Security**.
- May include utilities for notification, email services, and scheduling tasks.

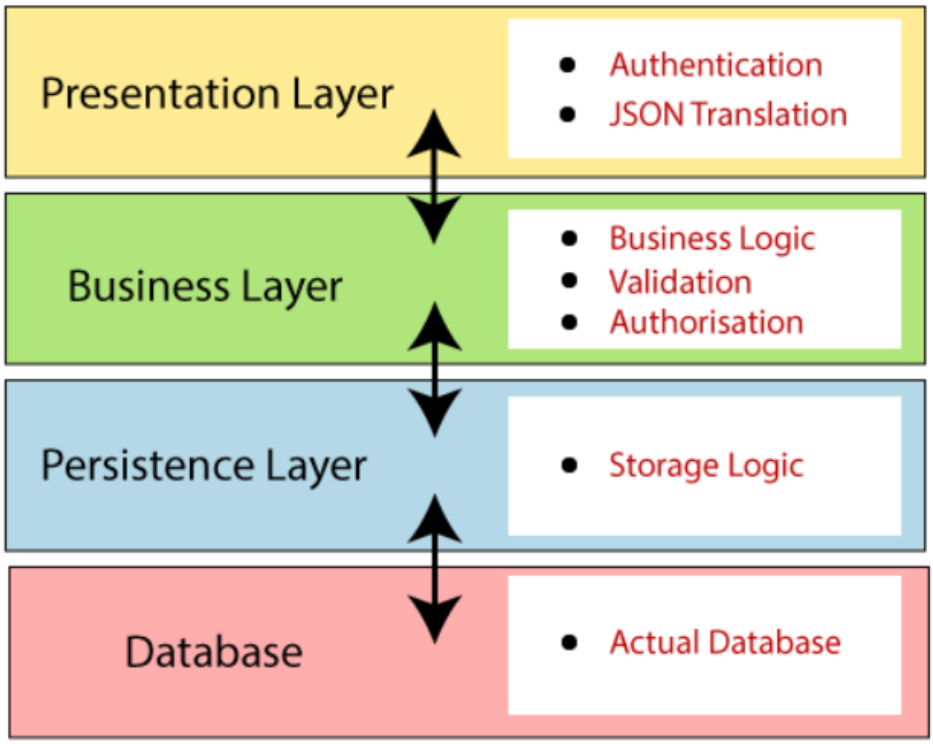


Figure 4.3: Layered Architecture

4.2.2.5 Benefits of Layered Architecture

- **Separation of Concerns:** Each layer has a distinct responsibility.
- **Scalability:** Easy to add new features or layers.
- **Maintainability:** Code is modular and easier to debug or extend.
- **Reusability:** Services can be reused in different parts of the application.

By applying this architecture, our Bank App remains robust, well-organized, and ready to evolve with future needs.

Chapter 5

Implementation

5.1 Introduction

The implementation and testing phase represents the final step in the development of our banking application. This chapter focuses on the tools and technologies used, key functionalities developed, and the designed user interfaces. We also discuss the unit tests performed to ensure the robustness of our application and showcase the graphical interfaces for administrators, bank staff, and customers.

5.2 Development Platform

5.2.1 Software Selection

Logo	Software	Reasons
	IntelliJ Ultimate	Comprehensive IDE for Java development with support for Spring Boot, JPA, and advanced debugging and refactoring tools.
	StarUML	Used to design UML diagrams (class, sequence, and use case) to model the structure and interactions within the banking system.
	XAMPP	Local development stack with MySQL used for database testing and backend integration.

Tableau 5.1: Software Used During Development

5.2.2 Front-end Technologies

- **HTML**: Used for structuring user interfaces like registration forms and transaction lists.



- **CSS**: Used to style components of the banking web portal (buttons, menus, input fields).



- **JavaScript**: Adds dynamic features such as form validation and real-time updates to the UI.



5.2.3 Back-end Frameworks

Spring Boot: Provides a production-ready environment for developing and deploying banking features like account creation, authentication, transfers, and admin tools.



Figure 5.1: Spring Boot

Spring Data JPA: Helps manage data related to users, transactions, and accounts with simplified repository-based access.



Figure 5.2: Spring Data JPA

5.2.4 Version Control and CI/CD

GitHub is used for version control, collaboration, and continuous integration. GitHub Actions automates build and test processes.



Figure 5.3: GitHub and CI/CD

5.3 Admin Panel: Client Management Overview

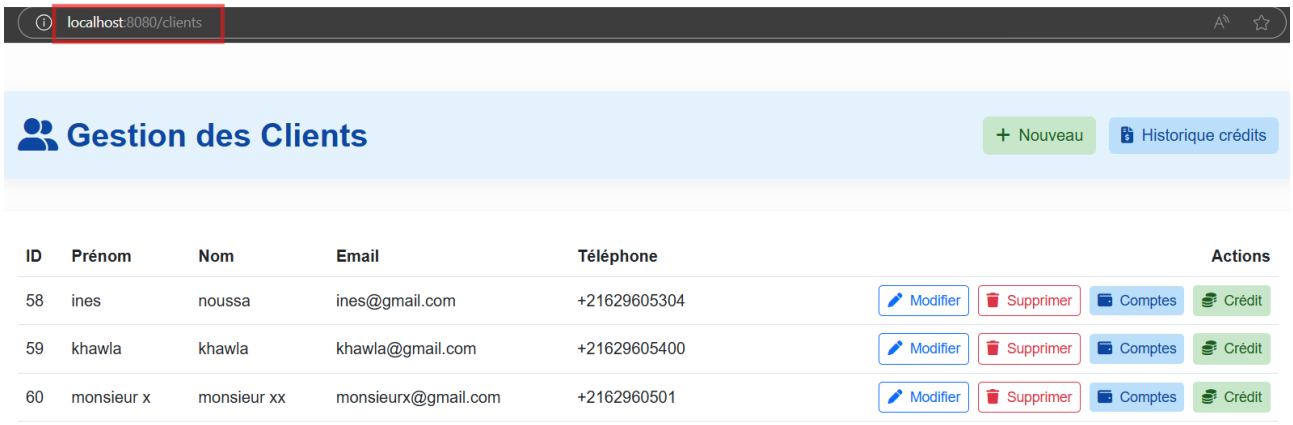


Figure 5.4: Admin Interface: Client Data Tablef

5.4 Client Information Editing Interface

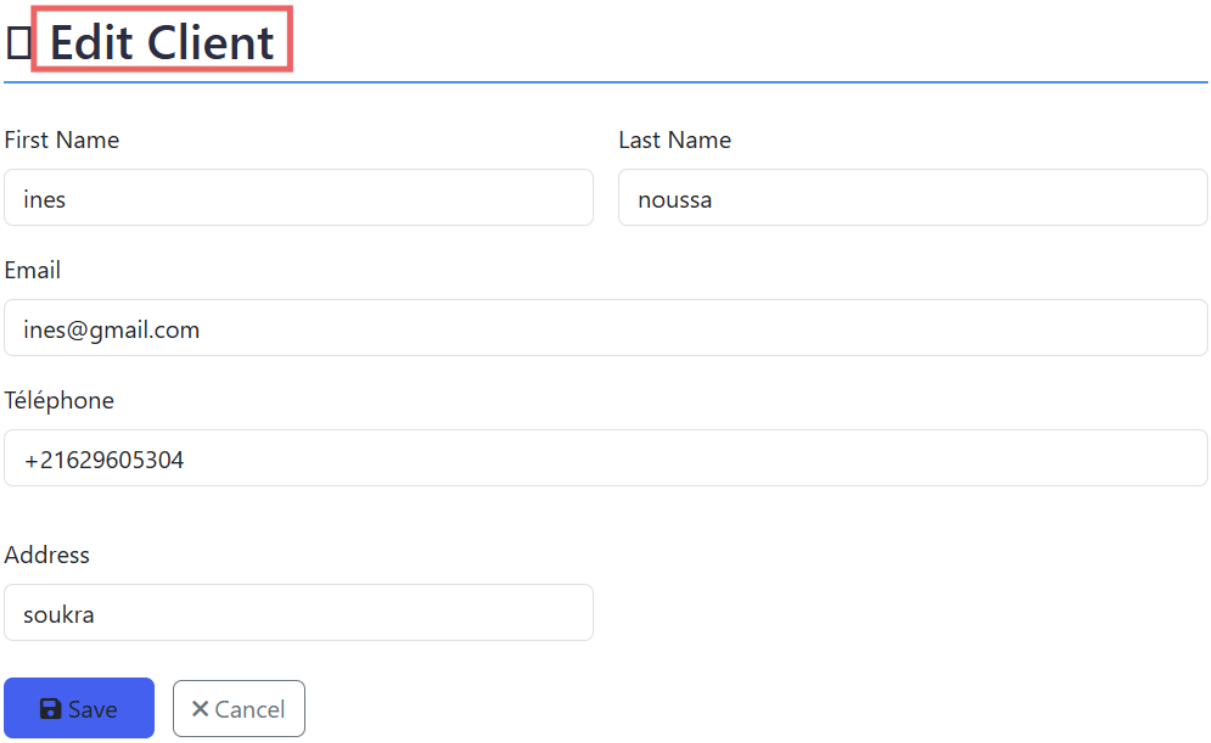


Figure 5.5: Edit Client Details Form

5.5 Client Deletion Confirmation Process

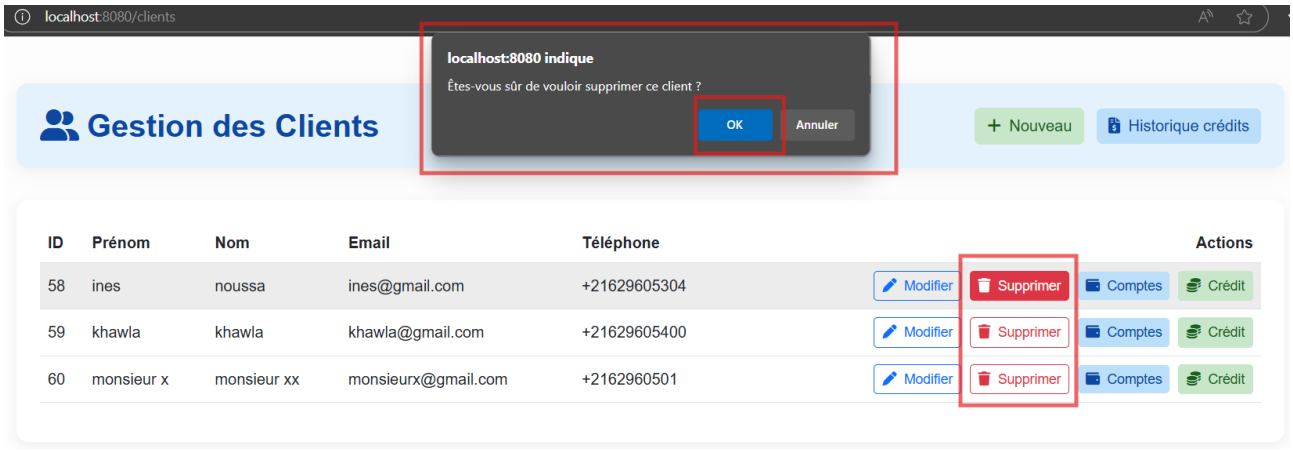


Figure 5.6: Client Removal Confirmation Dialog and List

5.6 Bank Account Management Dashboard



Figure 5.7: Bank Account List with 'View Transactions' and 'Create New Account' Options

5.7 Bank Account Operations Overview

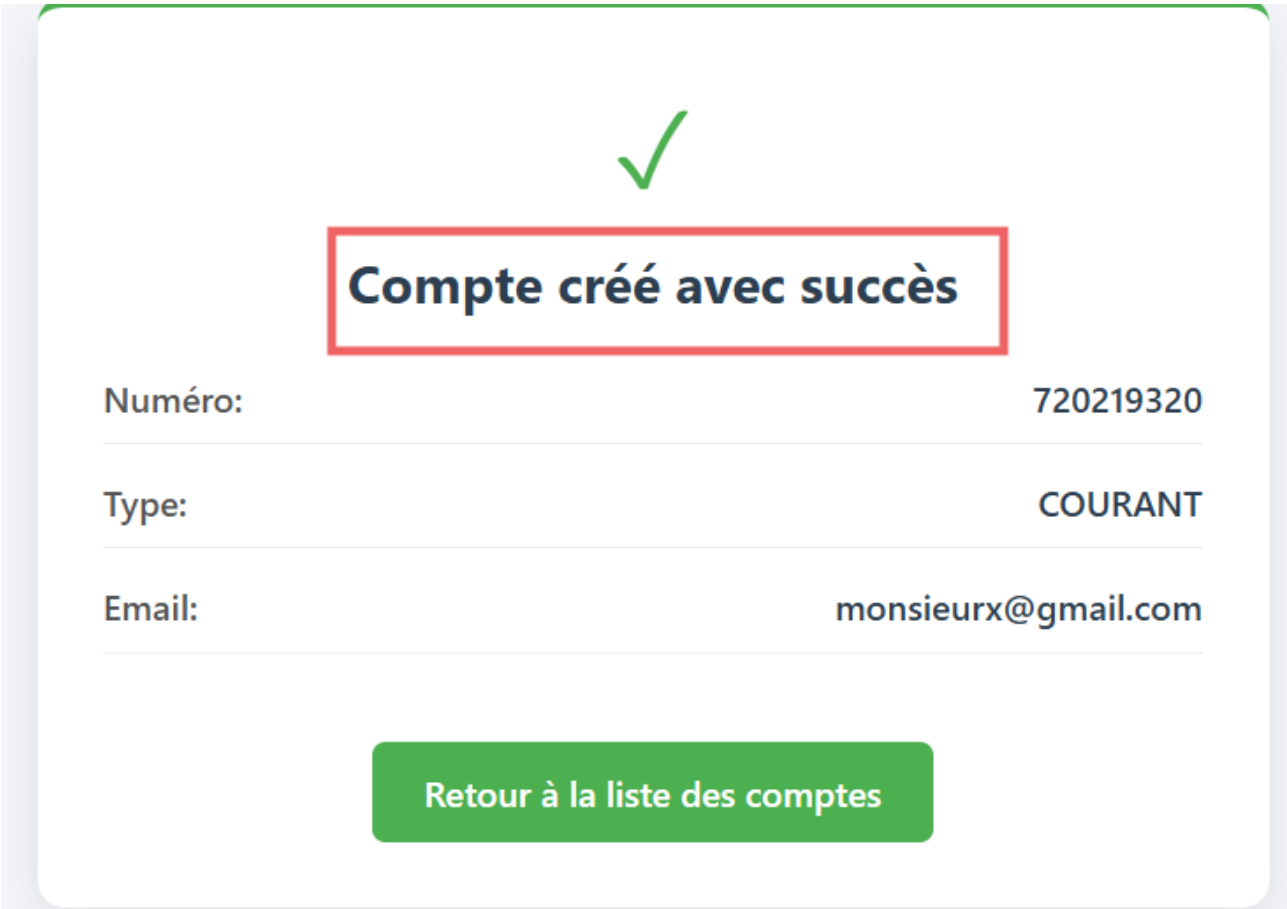


Figure 5.8: Bank Account Management Table with 'View Operations' Action

5.8 Transaction History and Account Activity

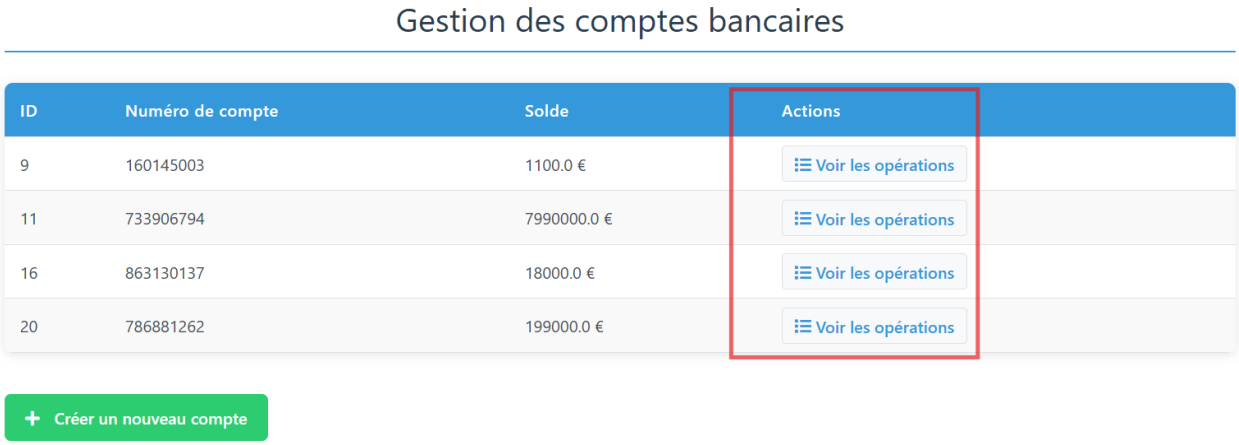


Figure 5.9: Bank Account Operations Overview

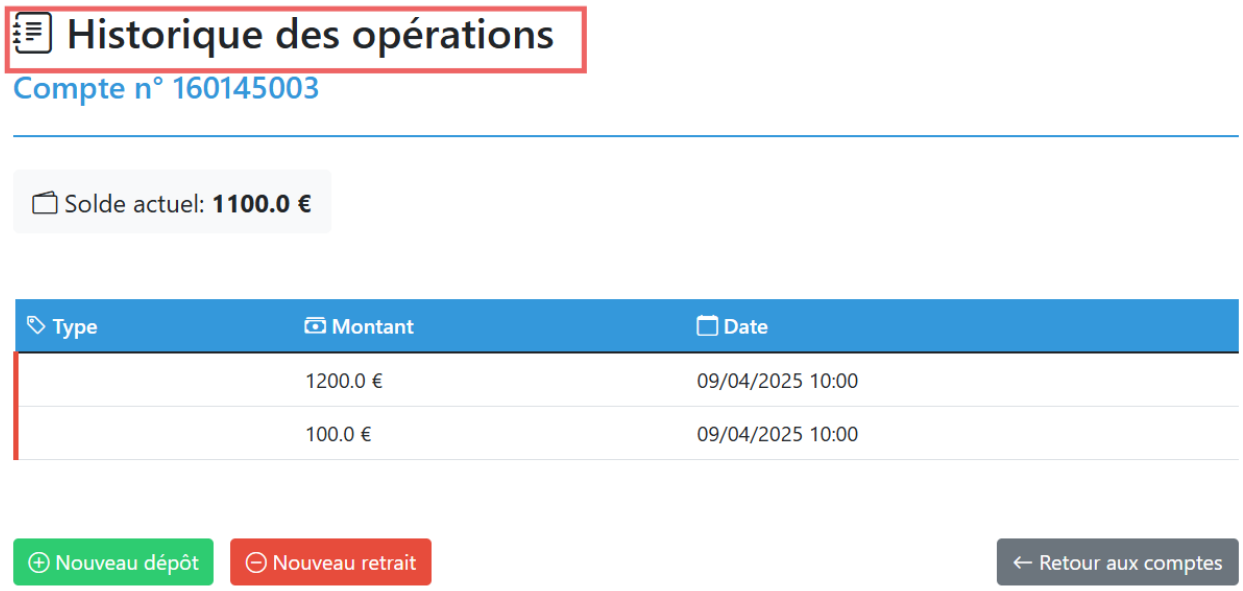


Figure 5.10: Bank Account Transaction History with Deposit/Withdrawal Options

5.9 Deposit and Withdrawal Transaction Interface

5.9.1 Deposit Transaction Interface

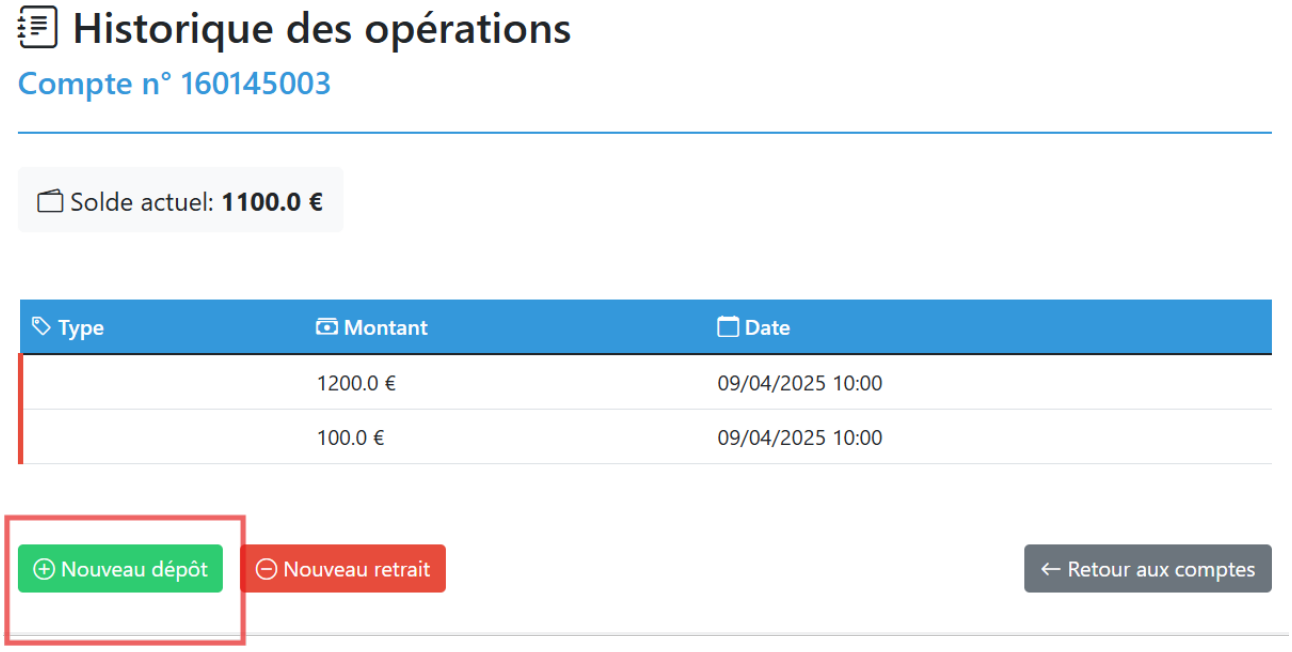


Figure 5.11: Bank Account Operations Overview

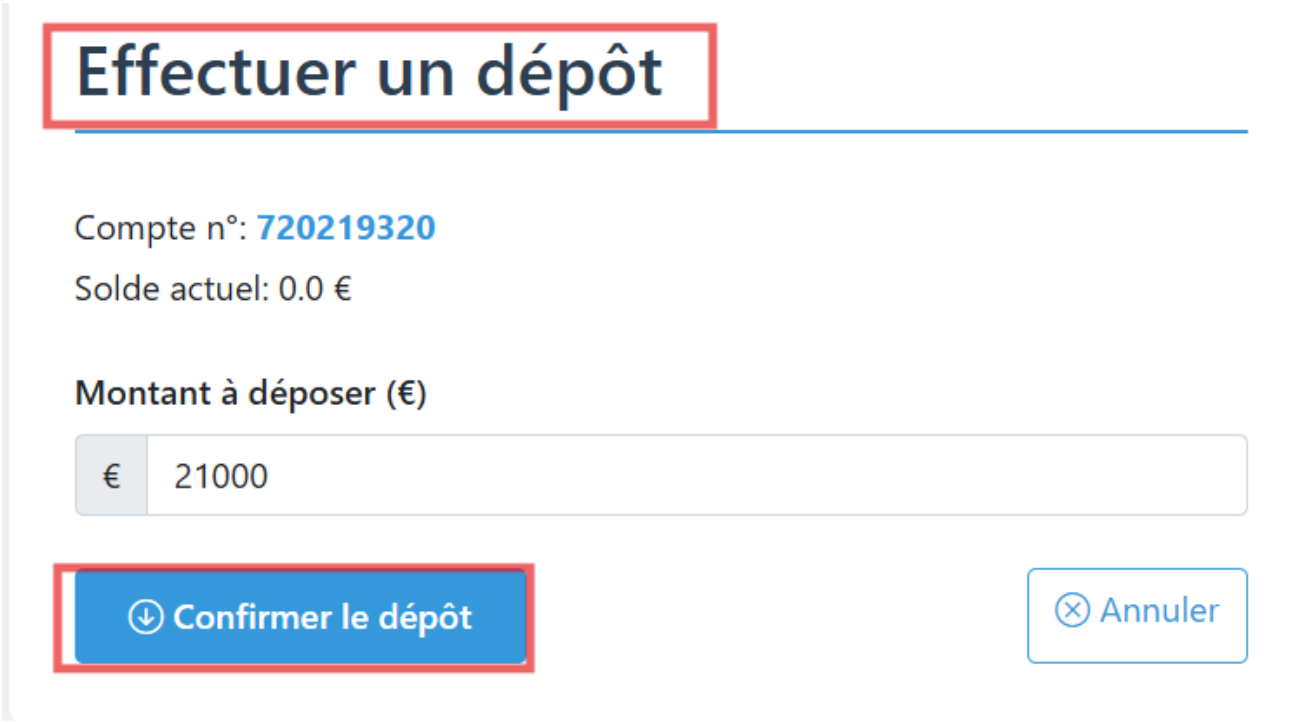


Figure 5.12: Bank Account Deposit Form with Confirmation Options

5.9.2 Withdrawal Operation Interface



Figure 5.13: Account Withdrawal Form with Amount Entry

5.10 Client Credit Management Dashboard

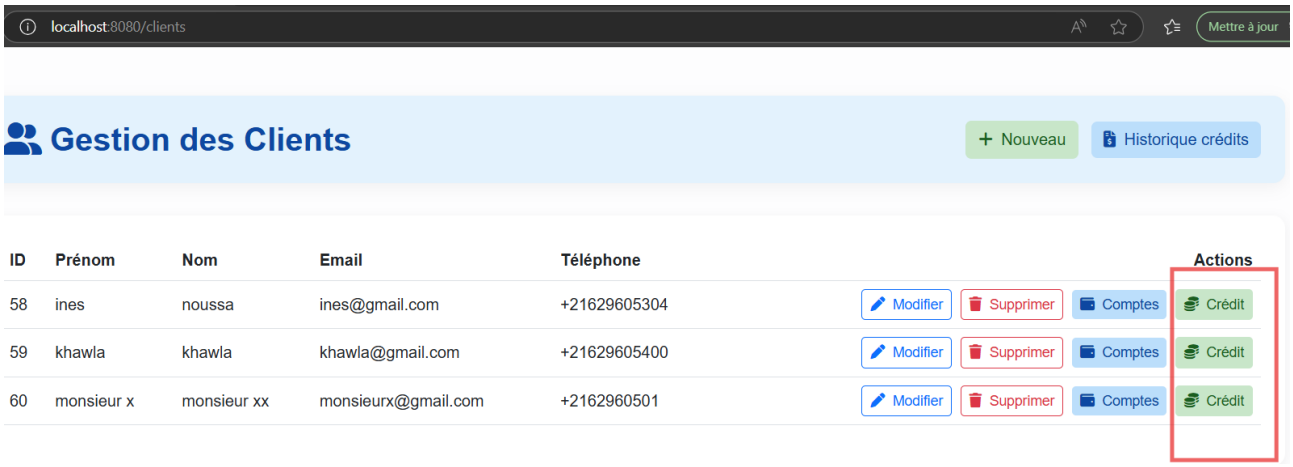


Figure 5.14: Client Credit Management Dashboard

5.11 Credit Application Submission Confirmation

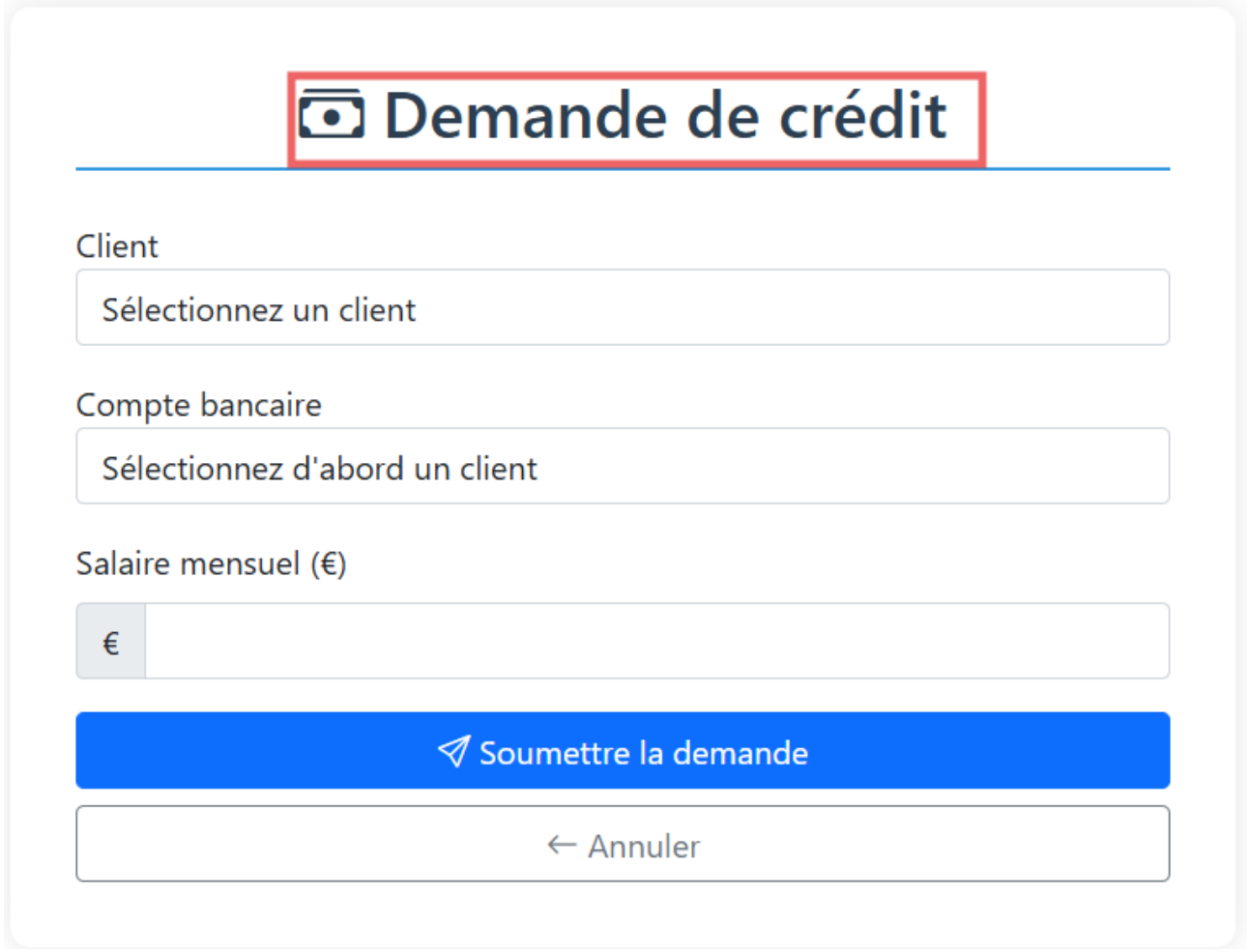


Figure 5.15: Successful Credit Request Receipt with Summary

5.12 Credit Application Submission Confirmation

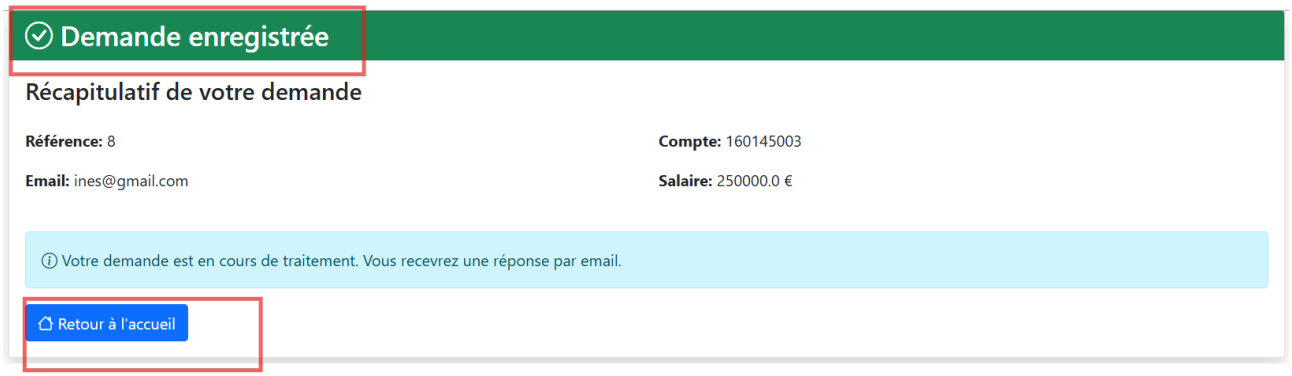


Figure 5.16: Successful Credit Request Receipt with Summary

5.13 Credit Application History and Status Tracking



Figure 5.17: Credit Request Log with Approval/Rejection Status

5.14 Bank Card Management System

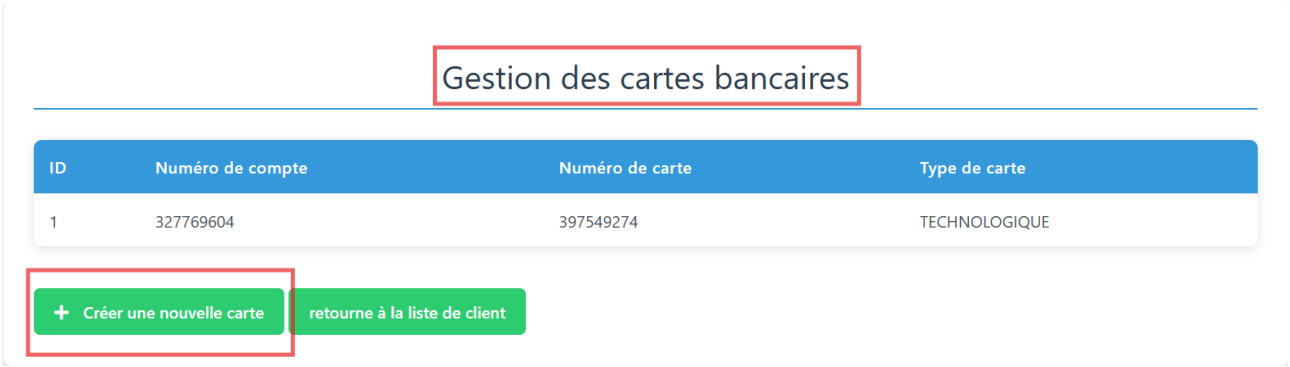


Figure 5.18: Bank Card Management Interface with Card Creation Option

5.15 New Bank Card Creation Interface

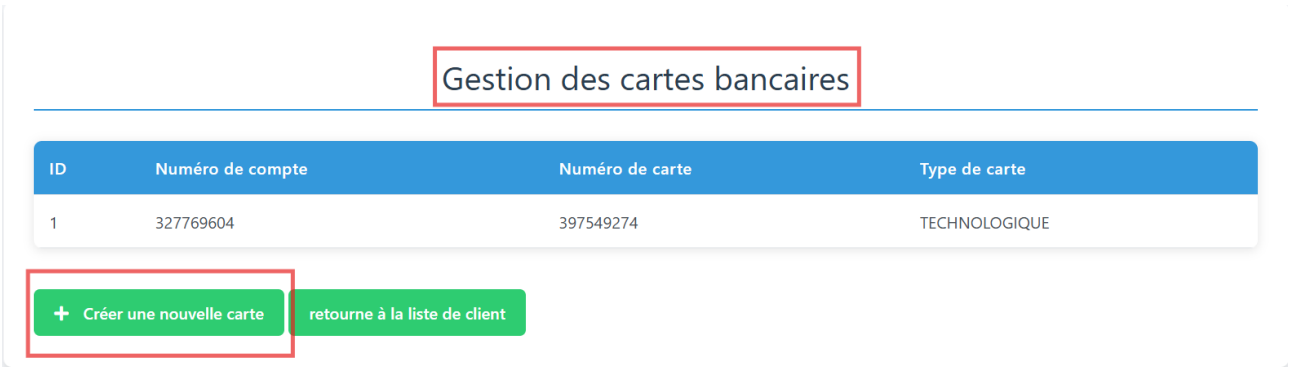


Figure 5.19: Bank Card Registration Form

5.16 Bank Card Creation Confirmation"



Figure 5.20: Successful Bank Card Generation Notification

General Conclusion

In this report, we have presented the development process of a secure and user-friendly banking application. Starting from the analysis of requirements and system design, through implementation using modern technologies such as Spring Boot, Spring Data JPA, and Thymeleaf, to testing and deployment, each phase was carefully carried out to ensure the reliability and efficiency of the final product.

The application allows customers to manage their accounts, perform transactions, and consult their financial history in a secure and intuitive interface. For bank administrators and staff, the platform offers management tools to monitor users, oversee operations, and ensure customer service quality.

The use of UML diagrams through StarUML helped structure the architecture and facilitated the planning of development tasks. GitHub was used for version control and team collaboration, ensuring the project evolved efficiently.

This project allowed us to apply our theoretical knowledge in a practical context, strengthen our skills in web development, and understand the challenges of developing a real-world banking system. It also emphasized the importance of security, usability, and maintainability in software development.

In the future, this application can be further enhanced by integrating features such as mobile support, two-factor authentication, or integration with external payment APIs, aiming to deliver a more comprehensive banking experience.